



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPARTMENT OF PROGRAMMING LANGUAGES AND COMPILERS

Detailed Research Plan: Migrating Spring Boot MVC Systems to Java 21 Virtual Threads

Supervisor:

Gregory Reynolds Morse
Department of Programming
Languages and Compilers

Author:

Hamed Mohamed
MSc in Computer Science (Software
Architecture)
Neptun code: DG1EWF

Budapest, 2026

Abstract

This research plan defines an empirical engineering study of migrating a legacy Spring Boot MVC application to Java 21 Virtual Threads. The study examines how intrinsic synchronization around blocking work affects throughput, latency, errors, and `jdk.VirtualThreadPinned` events, and which additional architecture-level risks must be audited before enabling Virtual Threads. A versioned demonstration application will be evaluated in platform-thread, naive-virtual-thread, and refactored-virtual-thread conditions using controlled k6 workloads and Java Flight Recorder. The primary artifact includes source, configuration, workload scripts, raw measurements, recordings, and analysis metadata. The expected contribution is a traceable migration procedure that distinguishes carrier-thread pinning from request-context and database-resource contention, together with evidence-bounded engineering guidelines. Results will be reported as a single-machine case study with explicit uncertainty and external-validity limits.

Keywords: Java Virtual Threads, Spring Boot, thread pinning, Java Flight Recorder, performance engineering

Contents

1	Research Aim and Questions	2
1.1	Motivation and scope	2
1.2	Research questions	2
1.3	Expected contribution	3
2	Related Work and Research Gap	4
3	Methodology and Research Design	5
3.1	Research mode and artifact	5
3.2	Experimental conditions	5
3.3	Workload and observability	6
3.4	Analysis and decision rules	6
3.5	Validity, risks, and fallback	7
4	Schedule and Reproducibility	8
5	AI Use and Submission Integrity	9

Chapter 1

Research Aim and Questions

1.1 Motivation and scope

Java 21 Virtual Threads preserve a conventional thread-per-request programming model while allowing many more concurrent blocking tasks than a fixed platform-thread pool. They do not remove synchronization, resource-pool limits, or architectural coupling. A legacy Spring Boot MVC application can therefore become slower or less predictable after migration if blocking code prevents Virtual Threads from unmounting or if downstream resources become saturated.

The approved topic registration places this work in the Computer Science (Software Architecture) track. The thesis will build an intentionally realistic Spring Boot MVC demonstration application, measure platform, naive Virtual Thread, and refactored Virtual Thread execution, and produce a practical migration checklist. No scope changes have been made since the approved topic registration.

1.2 Research questions

RQ1: How does an intrinsic synchronization mechanism surrounding blocking work affect the throughput, latency, error rate, and Virtual Thread pinning behaviour of a Spring Boot MVC application compared with platform threads and a Virtual-Thread-compatible lock?

RQ2: Which architectural patterns in a legacy Spring Boot MVC data path must be audited or refactored before enabling Virtual Threads, and what measurable risks remain after the primary pinning defect is removed?

The primary focus is intrinsic synchronization around blocking work. Manually managed `ThreadLocal` state and blocking JDBC access are treated as separate context-lifecycle and resource-contention risks, not as direct pinning causes unless JFR evidence supports that conclusion.

The bounded operational form of RQ2 is: in this demonstration application, how do intrinsic synchronization around blocking work, request-context handling with `ThreadLocal`, and JDBC connection-pool saturation affect pinning, context correctness, and finite-resource contention under Java 21 workloads? The study does not claim to identify every legacy anti-pattern.

1.3 Expected contribution

The study will contribute a controlled Spring Boot artifact, a traceable three-condition comparison, a JFR-based pinning diagnosis procedure, a mechanism-level anti-pattern classification, and migration guidelines tied to measured evidence and explicit limitations.

Chapter 2

Related Work and Research Gap

JEP 444 and Java 21 documentation establish the Virtual Thread execution model and pinning limitations. JEP 491 changes monitor-related pinning in JDK 24, so the primary claims here are explicitly Java-21-specific. Recent work covers framework integration, database-driven servers, Spring applications, reactive comparisons, and steady-state Java measurement; these studies show that workload, framework integration, and finite resources can reverse a simplistic “Virtual Threads are faster” conclusion. Spring Boot documentation describes framework configuration but does not rewrite application synchronization logic. Tomcat and HikariCP documentation establish server-thread and database-pool constraints that can confound performance comparisons. Classic work on thread-based and event-driven servers provides the broader concurrency context.

The unresolved gap addressed here is empirical and bounded: a reproducible mechanism-focused study of a legacy Spring MVC-style application in which a blocking synchronized path is migrated first naively and then refactored, while a clean endpoint and a data/context path provide controls. The study connects `jdk.VirtualThreadPinned` stack traces to application migration decisions instead of treating throughput change alone as causal evidence.

Chapter 3

Methodology and Research Design

3.1 Research mode and artifact

This is an empirical engineering/system study. The primary artifact is a versioned Spring Boot application with controlled workload paths, Maven build configuration, H2 data, k6 workload scripts, benchmark orchestration, JFR recordings, raw k6 JSON outputs, and analysis documents.

Table 3.1: Experimental endpoint roles

Endpoint	Role	Phenomenon
GET /orders	Slow data path	300 ms simulated blocking work, JDBC access, cleaned-up <code>ThreadLocal</code>
POST /payments	Pinning treatment	500 ms simulated external call, intrinsic synchronization or <code>ReentrantLock</code>
GET /products	Clean control	Fast JDBC query without intentionally introduced application anti-patterns

3.2 Experimental conditions

The independent variables are execution mode, payment-path synchronization implementation, load level, and endpoint role. The three main conditions are:

1. platform threads with the legacy synchronized implementation;
2. Virtual Threads with the same legacy implementation;

3. Virtual Threads after replacing intrinsic synchronization with `ReentrantLock`.

A platform-thread plus `ReentrantLock` cell and a small JDK 24/25 boundary run are preferred extensions, but neither may displace reproducible Java 21 measurements. Load levels are 50, 100, and 200 k6 virtual users. The local application uses H2 and simulated delays; PostgreSQL-specific conclusions will not be claimed unless that deployment is separately executed and recorded.

3.3 Workload and observability

The k6 workload ramps for 15 seconds, holds the target for 30 seconds, and ramp-down takes 10 seconds. Each virtual user invokes all three endpoints and then pauses briefly. Final measurements will also include endpoint-isolated runs to distinguish cross-endpoint interference from aggregate-workload artefacts.

Each run will record Java and Spring versions, machine information, Git commit, configuration, VU level, duration, throughput, p50/p95/p99 latency, timeout and error counts, the JFR recording, the exact extraction command, and stack-trace classification of `jdk.VirtualThreadPinned` events.

3.4 Analysis and decision rules

The primary outcomes are throughput, latency percentiles, error rate, and application-level pinning count and duration. A pilot will determine the final number of independent process-level repetitions and the precision target. Historical three-run sets are exploratory and do not justify strong bootstrap inference. Repeated results will be reported using run-level medians, interquartile ranges, effect sizes, and uncertainty intervals appropriate to the retained number of independent runs. p99 will not be replaced by maximum latency.

A change is material when it is at least a 20% throughput difference, a two-fold p95 latency difference, or a five-percentage-point error-rate difference at the same load. RQ1 is answered as a pinning-related degradation only when JFR identifies the application lock, the refactored condition differs in pinning, and the performance difference is reproducible under the same protocol. RQ2 classifies each pattern as a confirmed pinning source, resource/context risk, or not demonstrated.

3.5 Validity, risks, and fallback

Internal validity risks include mixed-endpoint interference, warm-up effects, and database capacity. Mitigations are endpoint-isolated runs, fixed warm-up/hold protocols, recorded configuration, and explicit timeout handling. ThreadLocal claims are limited because the demo removes the value in a `finally` block; no memory leak or pinning claim will be made without a separate experiment. One application and one machine limit external validity, so the contribution is framed as a mechanism-focused case study.

Historical benchmark narratives contain claims requiring reconciliation with raw artifacts. The fallback is to retain only traceable measurements and report unresolved items as limitations. A connection-pool sensitivity experiment may be added only if it is reproducible; otherwise it remains future work.

Chapter 4

Schedule and Reproducibility

Table 4.1: Research schedule and inspectable outputs

Period	Activity	Output or fallback
Week 1	Evidence freeze and correction	Run manifest and commit/configuration mapping
Week 2	Method hardening	Isolated workload and fixed JFR extraction procedure
Weeks 3–4	Baseline and naive reproduction	Validated platform and naive runs; narrow claims if needed
Week 5	Refactoring comparison	Refactored runs and stack-trace classification
Week 6	Sensitivity and synthesis	Optional pool analysis, RQ answers, migration checklist
Week 7 on-ward	Thesis drafting	Methodology, results, discussion, reproducibility package

The retained evidence package will include source and configuration versions, run manifests, raw k6 JSON, JFR files, extraction output, machine information, and analysis tables. Public open-source software and synthetic H2 records are used; no human participants, personal data, or confidential data are required.

Chapter 5

AI Use and Submission Integrity

The student remains responsible for the research questions, experimental design, code, interpretation, and submitted text. The supplied ELTE declaration requires selecting either no AI use or AI use and, in the latter case, recording the tool, purpose, how it was used, and thesis location. It also requires verification and proper citation of AI-assisted material, student responsibility for the content, and a signed declaration in the final package. The interaction log and declaration are submission artifacts, not experimental evidence.

Conclusion

This plan defines a bounded, reproducible engineering study of Virtual Thread migration in Spring Boot MVC. It will answer both research questions through controlled comparisons, JFR attribution, explicit decision rules, and evidence-preserving fallback paths. The final thesis will not claim universal Virtual Thread superiority; it will report which migration risks are demonstrated in the artifact and how confidently the findings transfer.